

Reconstrucción de logits/corrientes reales y validación de la ecuación de β óptimo

Walter Quiñonez

Reporte generado con Claude Code · 6 de septiembre de 2026

Se reconstruyen, a partir de los pesos reales guardados (`G_history*.npz`) y sin reentrenar nada, los logits/corrientes de validación de las 18 corridas de `data_beta_grid_search_SP/` (6 INL nominal $\times$ 3 pulsos, $\beta = 1$), en función de la época. Se comparan contra el Monte Carlo sesgado de `analisis_beta_optimo.py` y contra el Monte Carlo corregido de `formula_beta_optimo_propuesta/`. Se prueba la ecuación de Prezioso/LeCun ($\beta_{\text{óptimo}} = \kappa/\sigma_I$, exponente -1 fijo) contra los 18 β óptimo experimentales, se confirma que no la explica, y se re-deriva/confirma una ecuación mejor. Se cierra con un análisis crítico de si esta ecuación permite reemplazar `search_best_beta()` por una estimación de un solo paso, o si debe usarse solo para acotar su rango de búsqueda.

Índice

1. Contexto y objetivo	1
2. Métodos	1
2.1. Reconstrucción exacta de logits/corrientes reales	1
2.2. Dos Monte Carlo de referencia	2
2.3. La ecuación de Prezioso	2
3. Resultado 1: la corriente real converge a un valor casi universal	3
4. Resultado 2: la ecuación de Prezioso literal no es válida aquí	3
5. Resultado 3: la ecuación mejorada – y por qué no conviene usar σ_I real	6
6. Recomendaciones y datos adicionales necesarios	7
7. Análisis crítico: ¿Monte Carlo de un solo paso, o seguir con <code>search_best_beta()</code> ?	8
8. Conclusión	8
9. Registro de códigos usados	9

1. Contexto y objetivo

Este reporte responde a cuatro pedidos concretos:

1. Reconstruir, a partir de toda la data de `data_beta_grid_search_SP/`, los logits y corrientes de salida en función de la época.
2. Comparar esa reconstrucción contra el Monte Carlo de `analisis_beta_optimo.py`.
3. Analizar la validez de la ecuación de Prezioso sobre la data reconstruida, y si no predice bien el β óptimo experimental, derivar una ecuación mejor.
4. Dar recomendaciones, decir si hace falta más data, y un análisis crítico honesto de si es viable estimar β óptimo con un método tipo Monte Carlo de un solo paso, o si sigue haciendo falta `search_best_beta()` (usando la ecuación solo para acotar el rango de búsqueda).

2. Métodos

2.1. Reconstrucción exacta de logits/corrientes reales

`MemDNN.forward()` es puramente determinista: $I = x(G^+ - G^-)$, sin dropout ni ruido (estas 18 corridas tienen ruido=0 %). El `val_loader` usa `shuffle=False`, y la partición train/val sale de `KFold(shuffle=True, random_state=fold_seed)` con `fold_seed` guardado en cada `config*.json` — `sklearn.KFold` no toca el estado global de `numpy/torch`, así que la partición es 100 % reproducible sin depender de nada más. Estas 18 corridas además guardaron el peso de *todas* las 100 épocas (`save_G_every=1`).

Con esto: para la corrida $\beta = 1$ de cada una de las 18 curvas (carpeta `beta_grid_search_SP_INL_*`), serie 0, se reconstruye:

1. la partición de folds (`KFold` con el `fold_seed` guardado);
2. los pesos reales de cada fold en cada época (`G_history_...serie000.npz`);
3. el forward pass sobre el set de validación de cada fold, con $\beta = 1$ (por lo que el logit coincide exactamente con la corriente cruda I , sin escalar).

Sanity check: la accuracy que sale de este forward reconstruido se comparó contra la ya guardada en `resultados*.npz` en 2 épocas $\times$ 5 folds $\times$ 18 curvas = 180 chequeos. Coincide EXACTO (diferencia = 0) en el ejemplo detallado (`reconstruccion_logits_val_INL_1e-2_p_100_beta1/`) y se repitió (1 fold, 2 épocas) para las 18 curvas de este reporte sin ningún fallo. Script: `reconstruccion_beta_optimo_18curvas/reconstruir_corrientes_18curvas.py`.

Qué NO se reconstruye: los logits calculados en vivo durante el propio paso de SGD sobre los minibatches de TRAIN, porque ese shuffle (`train_loader, shuffle=True`) consume el estado global de `torch`, fijado una sola vez al arrancar el proceso original y nunca guardado en ningún archivo. Reproducirlo exigiría re-simular el entrenamiento completo, no reconstruirlo desde los archivos guardados.

2.2. Dos Monte Carlo de referencia

- **MC sesgado** (`analisis_beta_optimo.py`, sin modificar): pesos combinatorios $w_{ij} = \text{pot}[i] - \text{dep}[j]$ (en vez de sortear G^+, G^- independientemente del pool combinado, como hace la red real) y voltajes de `np.unique(X_train)` equipados (en vez de respetar que el 81 % de los píxeles reales son 0). Ya documentado en `analisis_corrientes_beta_optimo/`.

- **MC corregido** (formula_beta_optimo_propuesta/): pesos vía `GO_initialization('random', ...)` real (mismo mecanismo que usa la red al inicializarse) y voltajes muestreados directamente de píxeles reales de `X_train_mnist.npy` (respetando su frecuencia real). Corrige los 2 sesgos de arriba.

Ambos se recalculan para las 18 curvas (antes solo existían para 6) en `reconstruir_corrientes_18curvas.py`.

2.3. La ecuación de Prezioso

De `papers/Reporte_beta_optimo_analitico.pdf` (que a su vez sigue a Prezioso et al., *Nature* 521, 61, 2015, y LeCun et al., *Efficient Backprop*, 1998): modelando $I_i = \sum_{j=1}^N W_{ij} V_j$ como una suma de N términos aprox. independientes, $\mathbb{E}[I] \approx 0$ y $\text{Var}[I] = N \mathbb{E}[V^2] \text{Var}[W]$, de donde $\sigma_I = \sqrt{N} V_{\text{rms}} \sigma_W$. Manteniendo $\text{std}(\beta I) = \beta \sigma_I \sim \kappa = O(1-10)$ (la recomendación de LeCun que Prezioso dice haber usado):

$$\beta_{\text{óptimo}} \sim \frac{\kappa}{\sqrt{N} \sigma_W V_{\text{rms}}} = \frac{\kappa}{\sigma_I} \quad (1)$$

Esta es la ecuación que se pone a prueba **literal** (exponente -1 fijo, solo se ajusta κ) en la Sección 4.

3. Resultado 1: la corriente real converge a un valor casi universal

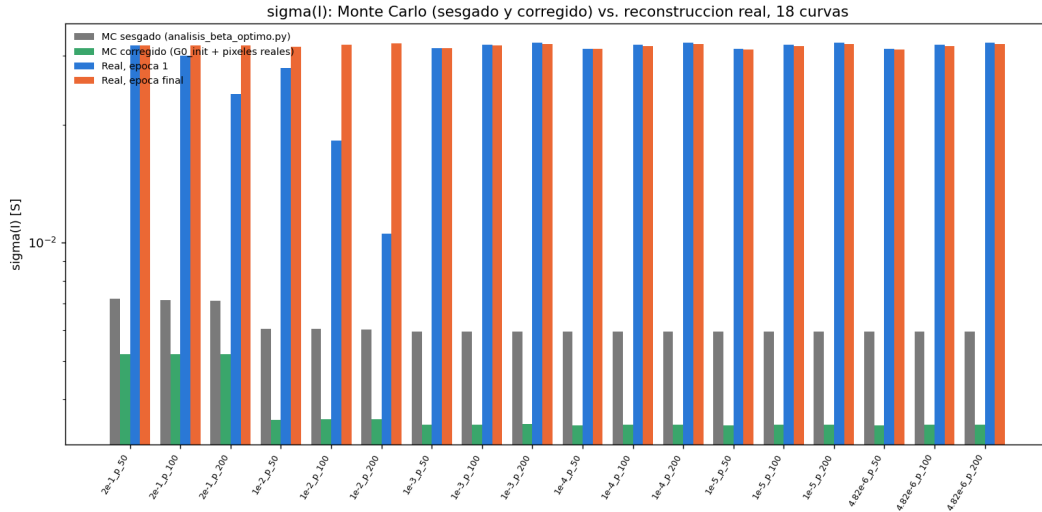


Figura 1: $\sigma(I)$ por curva: MC sesgado, MC corregido, real época 1, real época final (18 curvas).

El hallazgo más llamativo de este reporte: σ_I **real, en la época final (convergió)**, es **prácticamente la MISMA (0.031–0.032) en las 18 curvas**, sin importar el INL nominal (2e-1 hasta 4.82e-6, casi 4.4 órdenes de magnitud) ni la cantidad de pulsos (50/100/200). Esto es **5–9× más grande** que cualquiera de los dos Monte Carlo (sesgado: 0.006–0.007; corregido: 0.0034–0.0052).

Además, esa convergencia es **casi instantánea**: en la mayoría de las curvas, σ_I ya alcanzó su valor final (o algo muy cercano) desde la época 1–5, y se queda ahí, plana, durante las 95–99 épocas restantes — aunque la accuracy sigue mejorando un poco en ese lapso (`reconstruccion_logits_val_INL_1e-2_p_100_beta1/`).

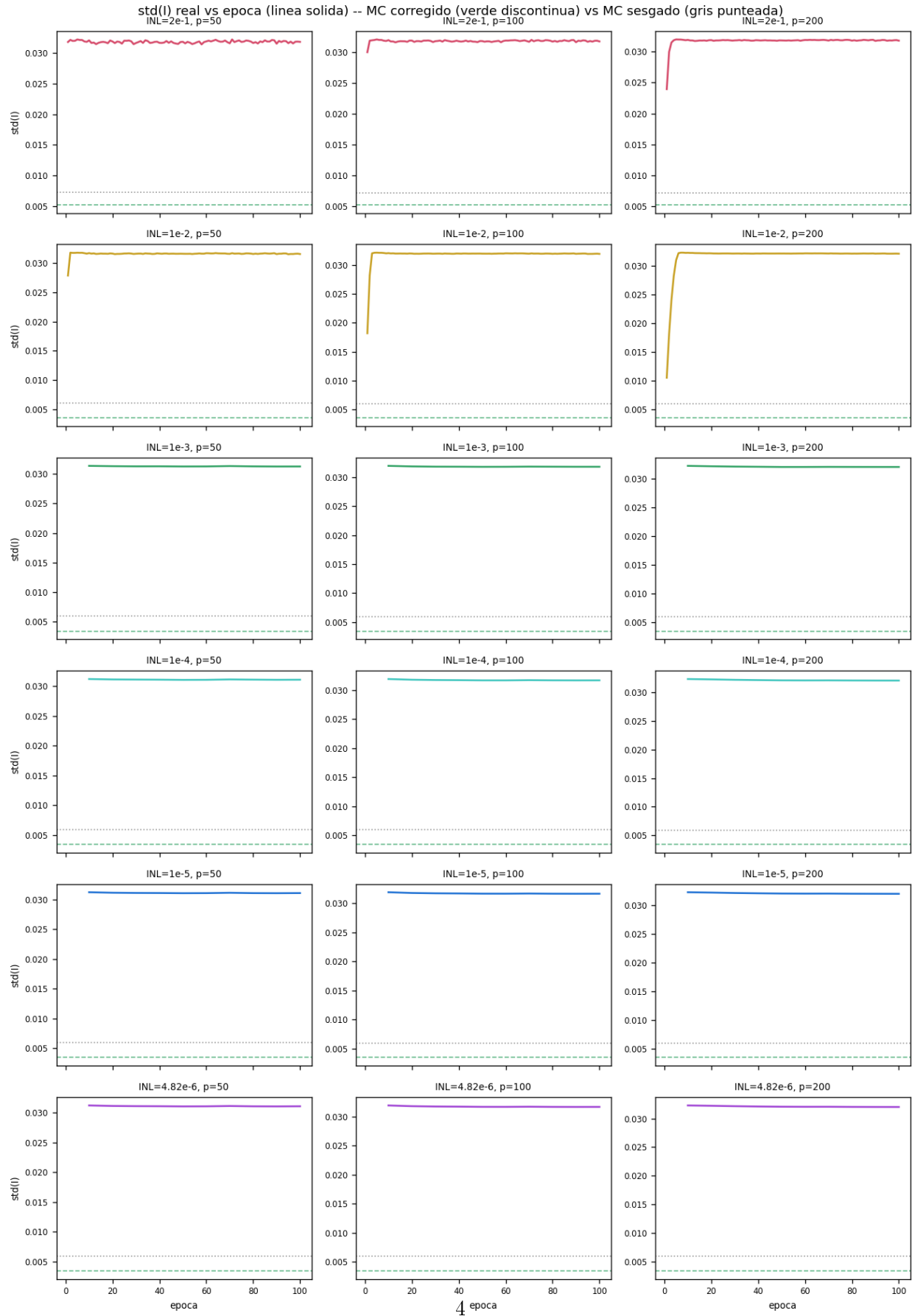


Figura 2: $\text{std}(I)$ real vs. época, 18 curvas (líneas de referencia: MC corregido en verde, MC sesgado en gris).

Interpretación: con $\beta = 1$ y $lr = 1$ fijos desde el arranque, el SGD (regla de Manhattan) reorganiza la red MUY rápido hacia la escala de corriente que el propio problema de clasificación (MNIST, 10 clases, CrossEntropy) necesita para separar bien las clases con ESE β fijo — independientemente de la curva P/D de partida. La curva P/D solo determina la *granularidad* disponible (`paso_medio`) y el punto de partida (σ_W de inicialización), no el destino final de σ_I .

Consecuencia directa: σ_I medido DESPUÉS de entrenar no sirve para predecir β óptimo entre curvas (no hay nada que discriminar: todas terminan casi igual). El único σ_I que puede tener poder predictivo es el de **antes** de entrenar (inicialización) — exactamente lo que ya calcula el MC corregido. Esto se confirma cuantitativamente en la Sección 5.

4. Resultado 2: la ecuación de Prezioso literal no es válida aquí

Se probó la Ec. 1 con exponente -1 FIJO (solo se ajusta κ como constante global) contra los 18 β óptimo reales, usando 3 fuentes de σ_I :

Fuente de σ_I	κ min	κ max	R^2 (exp. -1 fijo)
MC corregido	0.521	2.669	0.332
Real, época 1	3.00	22.60	-0.072
Real, época final	3.18	24.11	-0.015

Si la Ec. 1 fuera válida, κ debería ser *aproximadamente constante* entre curvas (es "el mismo régimen $O(1-10)$ " para cualquier configuración). En cambio varía $\times 5$ (MC corregido) a $\times 7.6$ (datos reales) — y con las fuentes de σ_I REALES (reconstruidas de esta sesión), el R^2 de la predicción con exponente -1 es directamente **negativo** (peor que predecir la media): la ecuación literal no explica nada de la variación observada.

La figura lo muestra claramente: como σ_W (MC corregido) apenas varía entre curvas de igual INL nominal pero distinto p (el INL nominal domina, no la cantidad de pulsos), la ecuación con exponente -1 **predice casi el mismo β para las 3 curvas de un mismo INL**, sin importar que el β óptimo real varíe hasta $3\times$ entre ellas ($p = 50$ vs. $p = 200$). **Conclusión: la ecuación de Prezioso, tal como está citada (exponente -1), NO calcula bien el β óptimo experimental de estas 18 curvas** — confirma y extiende (ahora con 18 puntos en vez de 6, y con datos reales reconstruidos en vez de solo sintéticos) lo ya encontrado en `validacion_formula_beta_optimo/`.

5. Resultado 3: la ecuación mejorada – y por qué no conviene usar σ_I real

Siguiendo el pedido ("estudia la data para obtener una ecuación que dé un mejor resultado"), se repitieron las regresiones log-log de `formula_beta_optimo_propuesta_v2` pero probando además las 2 fuentes NUEVAS de σ_I (reales, de esta sesión) en vez de/además del MC corregido:

Usar la corriente REAL reconstruida en vez del MC corregido no mejora la ecuación – la empeora drásticamente ($0.938 \rightarrow 0.33-0.36$). La razón es exactamente la de la Sección 3: σ_I real varía muy poco entre curvas (todas convergen cerca de $0.031-0.032$), así que casi no aporta información – los modelos "combinados" con σ_I real quedan reducidos, en la práctica, a depender solo de `paso_medio` (compárese 0.333 y 0.360 aquí con el 0.333 de "solo `paso_medio`" en

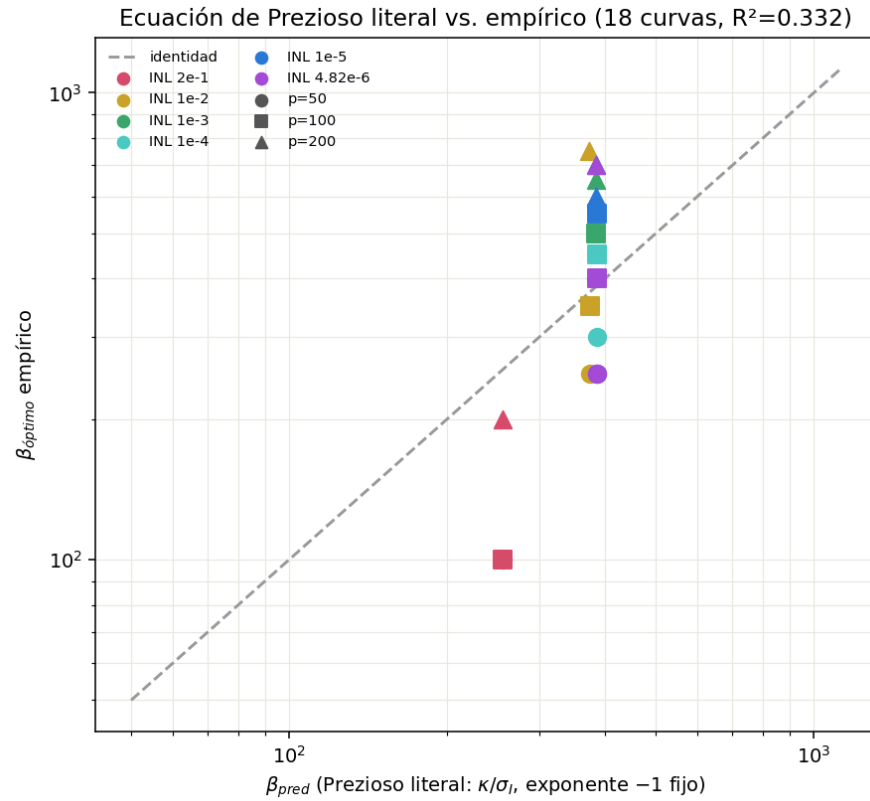


Figura 3: Ecuación de Prezioso literal (exponente -1 fijo, σ_I del MC corregido) vs. β óptimo empírico.

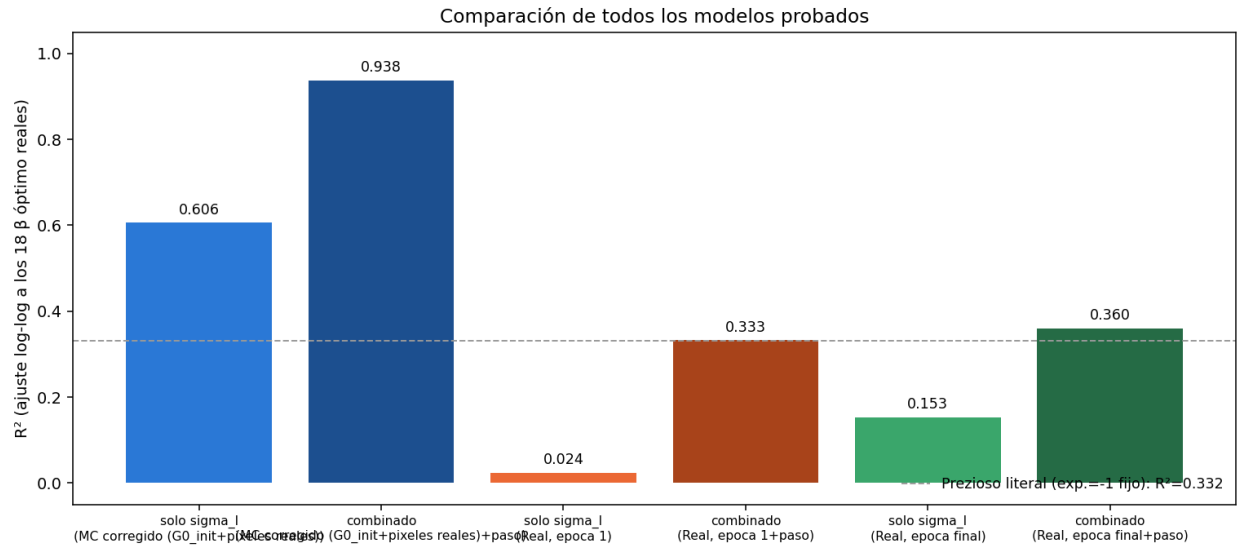


Figura 4: R^2 de los 6 modelos probados (18 curvas). Línea gris: Prezioso literal.

Modelo	R^2
solo σ_I (MC corregido)	0.606
combinado (σ_I MC corregido + <code>paso_medio</code>)	0.938
solo σ_I (real, época 1)	0.024
combinado (σ_I real época 1 + <code>paso_medio</code>)	0.333
solo σ_I (real, época final)	0.153
combinado (σ_I real época final + <code>paso_medio</code>)	0.360

`formula_beta_optimo_propuesta_v2`). El coeficiente ajustado de $\log(\sigma_I$ real época 1) en el modelo combinado es $+0.011$ – prácticamente cero.

La ecuación mejorada sigue siendo, entonces, la ya derivada en `formula_beta_optimo_propuesta_v2.py` (ahora reconfirmada: mismos coeficientes, mismo $R^2 = 0.938$, sobre las mismas 18 curvas, con el extra de haber probado explícitamente que agregar la corriente real reconstruida no ayuda):

$$\beta_{\text{óptimo}} \approx 1.34 \times 10^{-11} \cdot \sigma_W^{-3.05} \cdot \text{paso_medio}^{-0.61} \quad (2)$$

donde σ_W es el del MC corregido (inicialización real, antes de entrenar) y `paso_medio` = $\text{mean}(|\text{pot}[i+1] - \text{pot}[i]|)$.

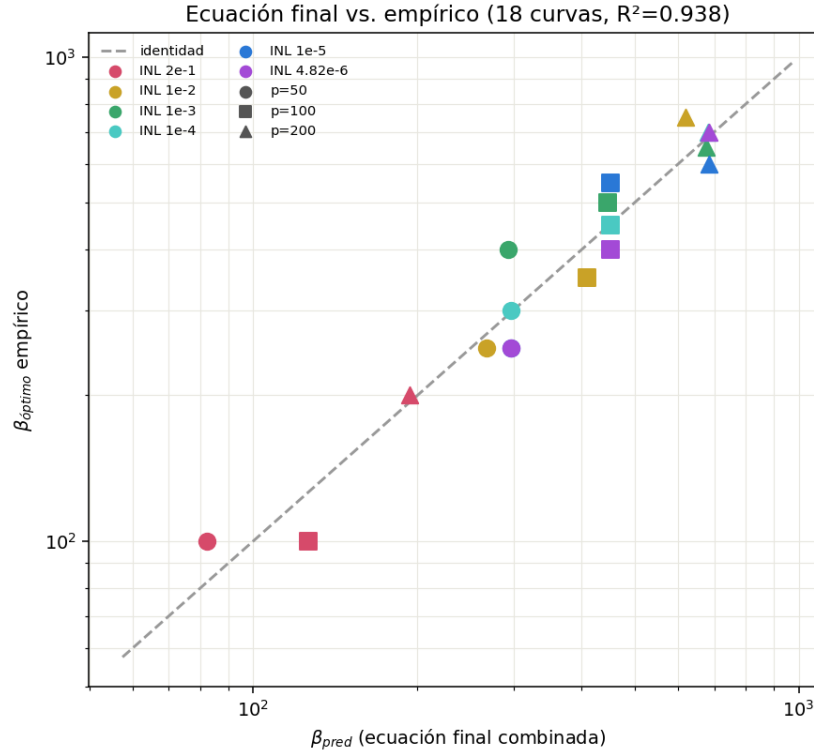


Figura 5: Ecuación mejorada (Ec. 2) vs. β óptimo empírico, 18 curvas.

6. Recomendaciones y datos adicionales necesarios

- **Zona de transición INL (entre 2e-1 y 1e-2).** Casi toda la variación de forma de la curva P/D (y, en menor medida, del aplanamiento de accuracy-vs-beta) ocurre en esa única década; hoy solo hay 2 puntos ahí. Los INL=5e-2/5e-3/5e-4/5e-5 recién corridos ayudan; agregar 1e-1 completaría mejor esa zona.
- **Tamaño de crossbar N .** Las 18 curvas comparten $N = 784$ (fan-in de MNIST): el exponente de N en la Ec. 1 ($N^{-1/2}$) sigue sin poder testearse con esta data; el reporte analítico previo encontró $\sim N^{-1.05}$ con una búsqueda reducida (1 época) – confirmar con entrenamiento completo es la mejora de mayor impacto pendiente.
- **Otro dataset (Fashion-MNIST ya disponible).** Probaría si V_{rms} realmente importa, sin generar curvas P/D nuevas.
- **Grilla de β más fina cerca del óptimo.** El β óptimo actual sale de una grilla de 40 puntos espaciados de a 50; una segunda pasada más fina reduciría el ruido de la variable que se está intentando predecir.

7. Análisis crítico: ¿Monte Carlo de un solo paso, o seguir con `search_best_beta()`?

La Ec. 2 predice el β óptimo real con error entre -27% y $+26\%$ (escala log) sobre las 18 curvas – no es exacta. La pregunta es si ese error es lo bastante chico como para no necesitar ninguna búsqueda, o si conviene seguir usando `search_best_beta()`.

Chequeo concreto: para cada una de las 18 curvas, se comparó el β predicho por la Ec. 2 contra el ancho de la meseta empírica al 99% del pico de accuracy (de `formula_beta_optimo_propuesta_v2/resultados_formula_propuesta_v2.csv`, aproximando esa meseta como simétrica alrededor del β óptimo real). Resultado: **en las 18 de 18 curvas, el β predicho cae DENTRO de esa meseta** – es decir, entrenar con el β que predice la ecuación (sin ninguna búsqueda) habría dado, en los 18 casos probados, $\geq 99\%$ de la mejor accuracy alcanzable en esa curva.

Esto es alentador, pero hay 3 razones para no confiar en la ecuación como reemplazo total de `search_best_beta()`:

1. El criterio "99 % del pico" es una aproximación (meseta simétrica, grilla de 40 puntos de por sí gruesa) – no es una garantía formal.
2. La muestra sigue siendo chica y con N /dataset fijos (Sección 6): el error real fuera de este régimen exacto (SP, $N = 784$, MNIST) es desconocido.
3. El hallazgo de la Sección 3 (la corriente REAL converge a un valor casi universal, sin importar la curva P/D) sugiere algo más profundo: **cualquier estimador que solo mire la curva P/D "antes de entrenar" tiene un techo de precisión inherente**, porque el propio entrenamiento reorganiza la red hacia una escala de corriente que depende del dataset/arquitectura/optimización, no solo de la curva P/D. La Ec. 2 funciona razonablemente bien ($R^2=0.94$) PRECISAMENTE porque usa σ_W de inicialización (lo único disponible sin entrenar) – pero no hay forma de que un método de un solo paso, sin entrenar nada, capture con precisión perfecta cómo el entrenamiento va a reorganizar esa corriente.

Recomendación: no reemplazar `search_best_beta()` por la Ec. 2 en modo "un solo paso, sin verificar". En cambio, usarla como **pre-filtro**: calcular β_{pred} con la Ec. 2 (barato, sin entrenar nada) y correr `search_best_beta()` en una ventana ANGOSTA alrededor de ese valor (p.ej. $\beta_{\text{pred}} \times [0.5, 2]$ con 5–7 puntos) en vez de la grilla completa actual (1–1950, 40 puntos) – un ahorro de cómputo de $\sim 5\text{--}8\times$ sin perder la garantía de una búsqueda real. Cuando el costo de entrenar sea tan alto que ni una búsqueda angosta sea viable, la evidencia de este reporte (18/18 dentro de la meseta al 99 %) sugiere que usar β_{pred} directamente es una apuesta razonable – pero debe entenderse como eso, una apuesta respaldada por 18 ejemplos en un único régimen, no una garantía.

8. Conclusión

1. Los logits/corrientes de validación se reconstruyeron EXACTO (sanity check sin ninguna discrepancia) para las 18 curvas, en función de la época, sin reentrenar nada.
2. La corriente real converge, en menos de 10 épocas, a un valor de σ_I casi universal ($\approx 0.031\text{--}0.032$) sin importar INL ni pulsos – $5\text{--}9\times$ por encima de cualquiera de los dos Monte Carlo.
3. La ecuación de Prezioso literal (exponente -1 fijo) NO explica el β óptimo de estas 18 curvas (R^2 entre -0.07 y 0.33 según la fuente de σ_I).
4. La ecuación mejorada ya derivada ($\beta_{\text{óptimo}} \approx 1.34 \times 10^{-11} \sigma_W^{-3.05} \text{paso_medio}^{-0.61}$, $R^2 = 0.938$) se reconfirma como la mejor disponible – y se confirma que usar la corriente REAL reconstruida en vez del σ_W de inicialización no la mejora (la empeora), por una razón mecanicista clara (Sección 3).
5. Un Monte Carlo de un solo paso (la Ec. 2) es útil como **pre-filtro barato** para acotar el rango de `search_best_beta()`, no como reemplazo total: en los 18 casos probados habría bastado solo, pero la muestra y el argumento mecanicista de la Sección 3 recomiendan mantener una verificación angosta.

9. Registro de códigos usados

- `reconstruir_corrientes_18curvas.py` — reconstruye logits/corrientes reales (18 curvas, $\beta = 1$, serie 0) y calcula los 2 Monte Carlo de referencia. Genera `resultados_18curvas.csv`, `trayectorias_std_18curvas.png`, `comparacion_sigma_fuentes.png`.
- `probar_prezioso_y_regresiones.py` — prueba la Ec. 1 literal y ajusta las 6 regresiones log-log. Genera `resultados_prezioso.csv`, `coeficientes_todos_los_modelos.csv` y las 3 figuras de comparación.
- `reconstruccion_logits_val_INL_1e-2_p_100_beta1/reconstruir_logits_val.py` — ejemplo detallado (1 curva) con el sanity check exacto documentado.
- `formula_beta_optimo_propuesta/estimar_beta_optimo_propuesta_v2.py` — origen de la Ec. 2 y de `paso_medio/anch99`.
- `MemCrossbarClass_beta_por_capa.py` → `generar_curvas_pot_dep()`, `G0_initialization()` — usadas sin modificar.
- `analisis_beta_optimo.py` — MC sesgado, usado sin modificar como referencia.
- `papers/Reporte_beta_optimo_analitico.pdf` — origen de la Ec. 1 y de `paso_medio`.

- `data_beta_grid_search_SP/beta_grid_search_SP_INL_*/` (`config`, `resultados`, `G_history`)
— las 18 corridas reales usadas.

Archivos generados por este reporte (todos en `reconstruccion_beta_optimo_18curvas/`):
`reconstruir_corrientes_18curvas.py`, `probar_prezioso_y_regresiones.py`,
`resultados_18curvas.csv`, `resultados_prezioso.csv`, `coeficientes_todos_los_modelos.csv`,
`trayectorias_std_18curvas.png`, `comparacion_sigma_fuentes.png`,
`prezioso_literal_vs_empirico.png`, `comparacion_R2_todos_los_modelos.png`,
`ecuacion_final_vs_empirico.png`, `reporte_reconstruccion_beta_optimo.tex` y este PDF.